

Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет
"Высшая школа экономики"»

Московский институт электроники и математики им.
А.Н. Тихонова НИУ ВШЭ
Департамент компьютерной инженерии

Курс: Алгоритмизация и программирование

ОТЧЕТ
по лабораторной работе №5

Раздел	Максимальная оценка	Итоговая оценка
Работа программы	1	
Тесты	1	
Правильность алгоритма	3	
Листинг	0,5	
Ответы на вопросы	2	
Дополнительное задание	3	
Итого		

Студент: Солодилов Михаил Анатольевич
Группа: БИВ253
Вариант: 232 (5, 12, 2)
Руководитель:
Оценка:
Дата сдачи:

Оглавление

Задание	2
Внешняя спецификация	3
Листинг программы	4
Распечатка тестов к программе и результатов	8

Задание

1. Создать файл для хранения действительных чисел, вводимых с клавиатуры. Прочитать этот файл и вычислить максимальное среди отрицательных чисел.
2. Создать связанный список для хранения целых чисел. Число записей неизвестно, но можно ввести число, являющееся признаком окончания ввода данных. Для четных номеров вариантов организовать очередь, а для нечетных – стек. Для исходного списка решить следующую задачу: вставить заданное число $A1$ после каждого элемента с нечетным номером.
3. Для полученного списка решить следующую задачу: упорядочить по убыванию методом «пузырька» элементы списка, расположенные после первого положительного элемента.

Внешняя спецификация

Лабораторная работа №5

Задание №1

Введите действительные числа, ввод закончите пустой строкой:
< x >

До пустой строки
При $max = -\infty$

В последовательности не было отрицательных чисел

Иначе

$max = \ll max \gg$

Задание №2

Введите целые числа, ввод закончите пустой строкой:
< n >

До пустой строки

Очередь:
 $\ll q[0] \gg, \ll q[1] \gg, \dots$

Введите A1:
< $A1$ >

Очередь:
 $\ll q[0] \gg, \ll q[1] \gg, \dots$

При $pos = NULL$

Нет положительных элементов

Иначе

Очередь:
 $\ll q[0] \gg, \ll q[1] \gg, \dots$

Листинг программы

```
1 #include <stdio.h>
2 #include <math.h>
3
4 static void read_stdin(const char* save_path);
5 static double find_max_neg(const char* path);
6
7 int main()
8 {
9     puts("Лабораторная работа №5");
10
11     puts("Задание №1");
12
13     read_stdin("data.txt");
14
15     double max = find_max_neg("data.txt");
16
17     if (max == -INFINITY)
18     {
19         puts("В последовательности не было отрицательных чисел");
20     }
21     else
22     {
23         printf("max = %lg\n", max);
24     }
25 }
26
27 static void read_stdin(const char* save_path)
28 {
29     FILE* save = fopen(save_path, "w");
30
31     char buffer[512] = "";
32
33     puts("Введите действительные числа, ввод закончите пустой строкой:");
34
35     fgets(buffer, sizeof(buffer), stdin);
36     while (buffer[1] != '\0')
37     {
38         double n = 0;
39         sscanf(buffer, "%lg", &n);
40         fprintf(save, "%lg\n", n);
41         fgets(buffer, sizeof(buffer), stdin);
42     }
43
44     fclose(save);
45 }
46
47 static double find_max_neg(const char* path)
48 {
49     FILE* f = fopen(path, "r");
50
51     char buffer[512] = "";
52
53     double max = -INFINITY;
54
55     while (fgets(buffer, sizeof(buffer), f))
56     {
57         double n = 0;
58         sscanf(buffer, "%lg", &n);
59
60         if (n < 0 && n > max)
61         {
62             max = n;
63         }
64     }
65
66     fclose(f);
67
68     return max;
69 }
```

```
1 #include <stdio.h>
2 #include <math.h>
3 #include <stdlib.h>
```

```

4
5 typedef struct Node
6 {
7     int data;
8     struct Node* next;
9 } Node;
10
11 typedef struct Queue
12 {
13     Node* head;
14     Node* tail;
15 } Queue;
16
17 static void push(Queue* q, int n);
18 static void insert(Node* after, int n);
19 static void queue_dtor(Queue* queue);
20 static void print_queue(const Queue queue);
21 static Queue input_queue(void);
22 static void insert_after_odd(Queue* queue, int a1);
23
24 static Node* find_positive(Queue queue);
25 static void sort(Node* node);
26
27 int main()
28 {
29     puts("Лабораторная работа №5");
30
31     puts("Задание №2");
32
33     Queue q = input_queue();
34     puts("Очередь:");
35     print_queue(q);
36
37     int a1 = 0;
38     puts("Введите A1:");
39     scanf("%d", &a1);
40     insert_after_odd(&q, a1);
41     puts("Очередь:");
42     print_queue(q);
43
44     Node* pos = find_positive(q);
45     if (!pos)
46     {
47         puts("Нет положительных элементов");
48     }
49     else
50     {
51         sort(pos);
52         puts("Очередь:");
53         print_queue(q);
54     }
55
56     queue_dtor(&q);
57 }
58
59 static void push(Queue* q, int n)
60 {
61     if (q->tail == NULL)
62     {
63         q->tail = calloc(1, sizeof(Node));
64         q->tail->data = n;
65         q->head = q->tail;
66         return;
67     }
68
69     Node* new_node = calloc(1, sizeof(Node));
70     new_node->data = n;
71
72     q->tail->next = new_node;
73     q->tail = new_node;
74 }
75
76 static void insert(Node* after, int n)
77 {
78     Node* new_node = calloc(1, sizeof(Node));
79     new_node->data = n;

```

```

80     new_node->next = after->next;
81     after->next = new_node;
82 }
83
84 static void queue_dtor(Queue* queue)
85 {
86     Node* cur = queue->head;
87
88     while (cur)
89     {
90         Node* next = cur->next;
91         free(cur);
92         cur = next;
93     }
94
95     *queue = (Queue){};
96 }
97
98 static void print_queue(const Queue queue)
99 {
100     Node* cur = queue.head;
101
102     while (cur)
103     {
104         Node* next = cur->next;
105         printf("%d\n", cur->data);
106         cur = next;
107     }
108 }
109
110 static Queue input_queue(void)
111 {
112     Queue q = {};
113
114     char buffer[512] = "";
115
116     puts("Введите целые числа, ввод закончите пустой строкой:");
117
118     fgets(buffer, sizeof(buffer), stdin);
119     while (buffer[1] != '\0')
120     {
121         int n = 0;
122         sscanf(buffer, "%d", &n);
123         push(&q, n);
124         fgets(buffer, sizeof(buffer), stdin);
125     }
126
127     return q;
128 }
129
130 static void insert_after_odd(Queue* queue, int a1)
131 {
132     size_t n = 1;
133     Node* cur = queue->head;
134
135     while (cur)
136     {
137         Node* next = cur->next;
138         if (n % 2 == 1)
139         {
140             insert(cur, a1);
141         }
142         cur = next;
143         n++;
144     }
145 }
146
147 static Node* find_positive(Queue queue)
148 {
149     Node* cur = queue.head;
150
151     while (cur)
152     {
153         if (cur->data > 0)
154         {
155             return cur;

```

```

156         }
157         cur = cur->next;
158     }
159
160     return NULL;
161 }
162
163 static void sort(Node* node)
164 {
165     if (!node)
166     {
167         return;
168     }
169
170     Node* n1 = node;
171
172     while (n1)
173     {
174         Node* n2 = n1->next;
175         while (n2)
176         {
177             if (n1->data < n2->data)
178             {
179                 int t = n2->data;
180                 n2->data = n1->data;
181                 n1->data = t;
182             }
183             n2 = n2->next;
184         }
185         n1 = n1->next;
186     }
187 }

```


Распечатка тестов к программе и результатов

№	Исходные данные	Результат
1	$k = 1, S[0 : 0] = [\text{abba}]$	Не нашлось подстрок, ограниченных точками Не нашлось строки со скобками и цифрами
2	$k = 2, S[0 : 2] = [\text{y..b..c ..d, .dog при(1-2)ветfe.}]$	Нашлось 3 подстрок, ограниченных точками: b c dog при(1-2)ветfe Нашлась строка со скобками и цифрами: "dog при(1-2)ветfe" После очистки букв не из русского алфавита: "привет"
3	$k = 1, S[0 : 0] = [\text{.adas.dog приветfeline.}]$	adas dog приветfeline Не нашлось строки со скобками и цифрами